

COEN/ELEC 390 Scrum Assignment 2

Example

SensorTicTagToe App

Member 1 ID

Member 2 ID

Member 3 ID

Abstract— The Document Template presents a skeleton of a scrum assignment document for the SensorTag TicTacToe app, primarily showing diagram examples and describing the necessary full-text components for the document. *The content is intended to give an example on how to document the application design. However, this is not a static way of documenting your design, in some cases you will need to describe more details in order for the reader to understand your design and implementation. This is not a complete document of the SensorTicTagToe application, it is intended to be an explanation and guide with a real app example.*

Table of Contents

1	Introduction	1
1.1	Definitions, Acronyms, and Abbreviations	1
2	Requirements: Product Backlog	2
3	Design	4
3.1	Android Application Wireframes	4
3.2	System Architecture	4
3.3	Hardware Architecture	5
3.4	Software Architecture	5
3.5	Communication Interfaces	7
3.6	Use Cases and Sequence Diagrams	9
4	Testing	10
4.1	Test Plan 1: Start a new game	10
4.1.1	Summary	10
4.1.2	Test Cases	10
5	Sprint 2 Backlog	12
6	References	13
	Appendix A:	14

List of Figures

Figure 1	Wireframe of tictactoe app	4
Figure 2	A general system structure showing MVC design	5
Figure 3:	System structure (package diagram)	6
Figure 4:	Sequence diagram: player shakes SensorTag and successfully plays a piece	9

List of Tables

Table 1	Definition and Abbreviation	1
---------	-----------------------------------	---

1 Introduction

SensorTag TicTacToe is an android mobile application that allows one or two players to play TicTacToe using the SensorTag. The SensorTag is used for the player to navigate through the 3x3 grid and choose the position of his/her next move.

This application is intended for developers to easily implement their own TicTacToe AI and compete against each other.

1.1 Definitions, Acronyms, and Abbreviations

The following table details the specific terms and abbreviations used throughout this report.

Table 1 Definition and Abbreviation

Abbreviation	Term	Meaning
API	Application Program Interface	Set of protocols on the software to specify how components should interact.
GUI	Graphical User Interface	Type of user interface that allows the user to interact with graphical icons such as buttons on screen or text fields on screen.

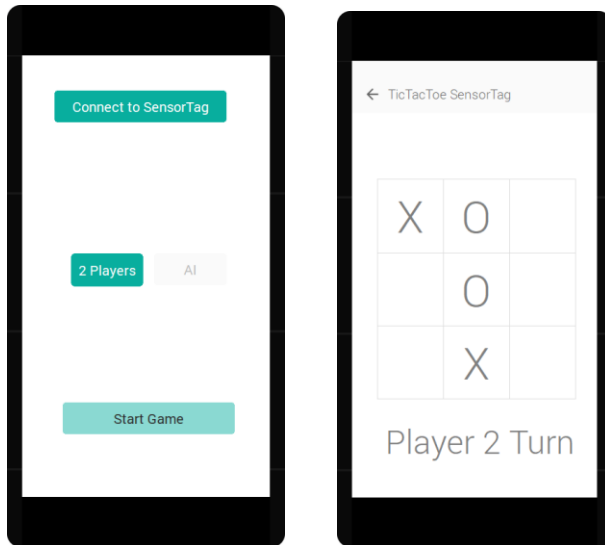
2 Requirements: Product Backlog

Story ID	Story Title	Card	Story Points	Sprint	Status	Conversation	Confirmation
HW-1	Connect to the SensorTag	<i>As a typical user I want the application to connect to SensorTag so that I can use the arrow buttons to navigate through the game grid.</i>	3	Sprint 1	Done	The connection to SensorTag should happen when the application is launched.	1. Is the application connected to SensorTag on launch? 2. On restart of application, does SensorTag connect automatically to the application?
GEN-1	1 or 2 players.	<i>As a typical user I want the ability to select my opponent so that I can choose to play a human or AI.</i>	5	Sprint 1	In Progress	This should be when the app is first launched. And the choice of the user should be saved temporarily incase the user restarts the game, it won't ask again.	1. can the user easily choose to play against an AI or a human? 2. when the user restarts the game, does the choice of AI or human stays the same?
GEN-2	TicTacToe game implementation.	<i>As a typical user I want the application to follow all game rules so that I can enjoy the game in an interactive way.</i>	7	Sprint 1	Done	This will allow the user to play TicTacToe game or build an AI to play the game. The data structure should be simple and easy to	1. Can two players play tictactoe? 2. Can a user place a move at a spot already taken? 3. Can a developer easily build a class inheriting

						understand for any developer to use to build a custom AI for the game. Refer to <link here> for all game rules	from the main player class?
GEN-3	AI	<i>As an advanced user I want the ability to select AI as opponent so that I can use it as single player game.</i>	9	Future		The AI will follow a minmax design with a heuristic function more informative than just win or lose.	Can a human win against the AI? The AI should either win or draw.
UI-1	Cool UI	<i>As a typical user I want the application to have simple and attractive user interface so that the game is enjoyed by users of all age group.</i>	5	Future		The colors used should be light as the users will interact with the game for some time while playing. Refer to <link here> for application UI/UX	Would a kid enjoy playing the game?

3 Design

3.1 Android Application Wireframes



At the beginning the user will choose whether with the game 2 Players or against as AI then when the Start Game button is pressed the game will start.

Note: JustInMind is a very good wireframing software and it is free.

<https://www.justinmind.com/>

Figure 1 Wireframe of tictactoe app.

3.2 System Architecture

Since we are using Android, we are confined to the architecture that Android defines for us: the first level of architecture is therefore the Activities and then from there we have your own modules that interact with the Android components. The SensorTicTagToe application uses the sensor tag for the user to navigate through the 3x3 grid and choose a position for their move. It also communicates with the database on the phone to save the score of the player (wins, loses and draws). Figure 2 shows the system structure and the components this application uses.

Note: the MVC structure will be explained in one of the tutorials.

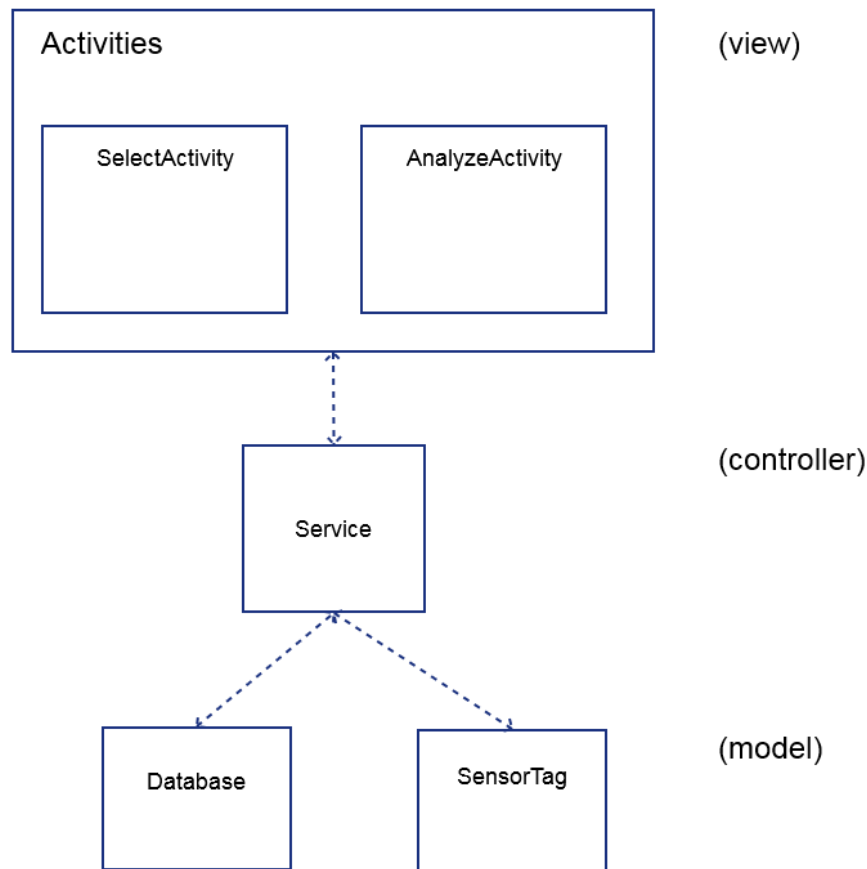


Figure 2 A general system structure showing MVC design.

3.3 Hardware Architecture

In this application we only use the SensorTag, in this example you would give a brief explanation to what the sensortag is and what sensors in it are you using and how are you connecting it to the android application. You would also mention the way of communication that is happening.

For example, in this case we are using the arrow buttons on the sensor. You would explain briefly what happens when the listener of the button gets calls onClick().

3.4 Software Architecture

For the SensorTicTagToe application, there is a TicTacToeGame class, which contains the game rules separately from the Activity. Figure 3 shows the software structure of the SensorTicTagToe app. It is an informal diagram that shows the main parts, their relations and interactions in general. In this case the blocks correspond roughly to “modules” which are usually collections of related classes but may occasionally represent a single class.

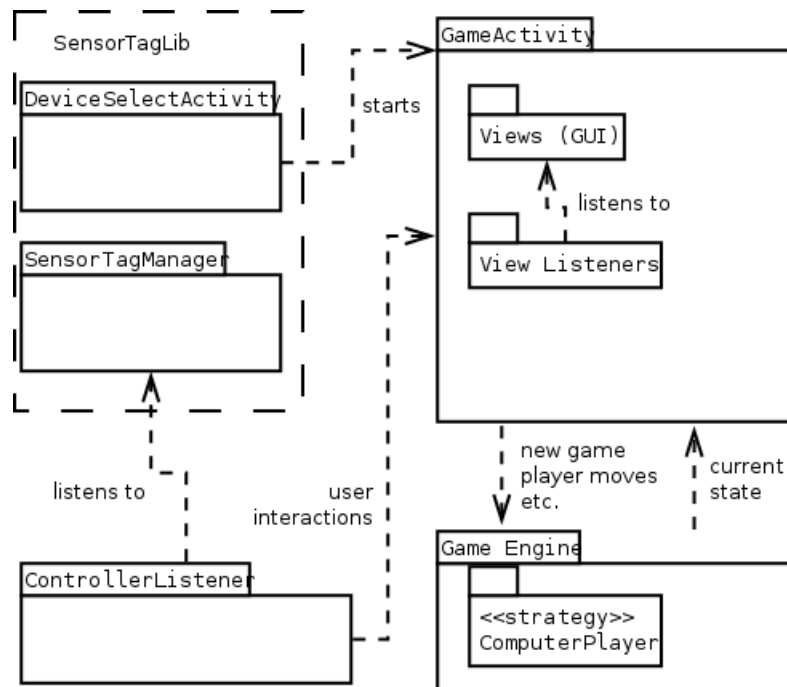


Figure 3: System structure (package diagram)

You will note that one design detail, the “Strategy” design pattern for the ComputerPlayer interface, is shown: this is because this interface is fairly critical to how Game Engine works (it is one of those *hard to change* elements—more in a big application than in this case) and needs to be decided early in order for different computer AIs to be possible and well designed.

Explain each module and, generally speaking, what its role is and relationships to other modules are. In this case, we showed a mixture of interactions (listens to, configure) and data flow (user interactions, new game, current state, etc.)—this is a very informal use of arrows.

Here’s an example of how you might describe this in your design document (you could also choose to use a table):

DeviceSelectActivity is from the SensorTagLib, and allows the user to scan for and select a SensorTag device to use as a controller. It is responsible for starting the Game Activity once a device has been selected.

SensorTagManager, also part of SensorTagLib, manages all setup and interactions with the SensorTag device. It presents a friendly API for the app to manage the enabled sensors and receiving data, and abstracts away all the Bluetooth connection details. See the SensorTagLib documentation [1] for an API description.

GameActivity is the main activity of the app, which shows the tic tac toe game interface and allows the user to play through a full game. It is responsible primarily for managing the GUI

and all related logic. It is also the central adapter between SensorTagManager, ControllerListener and GameEngine, ensuring that these components are created and configured correctly and that events from each one is communicated to the others as needed (e.g. a ControllerListener event to play a move is passed to GameEngine).

The **ControllerListener** listens to sensor data from the SensorTagManager, and processes it to detect user interaction events, which are then passed to GameActivity.

The **GameEngine** is responsible for keeping track of the current game board, game state (e.g. whose turn it is), and game rules (e.g. winning, ties, valid moves). It is also responsible for sending game state and board data back to the GameActivity to display on the GUI and determine valid interactions. Furthermore, the Game Engine can execute an AI player using a plugin-like Strategy design pattern.

3.5 Communication Interfaces

This section applies if your software communicates with the outside world. It defines how this communication occurs (for example, if you have a client or server component, or if you need to communicate between your Activity and a third-party Service using Intents, or if you communicate with an external service like Twitter).

Note that this section is not part of the Scrum Assignment Guide document, however multiple projects might need to mention it if they are using communication protocols and third-party services.

If you are using a custom API, you should define all the details of your API, as shown below. This is the case if you are writing your own client and server communications, or tailoring an existing protocol to your own specific needs (e.g. using an HTTP server and custom PHP script to receive commands or send data from a server).

If, however, you are using pre-existing protocols/API (e.g. Twitter API, plain HTTP to get data from a website), you can simply mention and reference that protocol/API in this section.

If you need to describe a custom API, see below:

The SensorTicTagToe does not have any communication interfaces (so this section would not be included for it). In this section, if you include it, you would define:

- The goal of each communication interface: for example, perhaps your system sends messages over the Internet in order to send and receive moves, in order to play tic tac toe online.
- How the communication happens. For example, via TCP/IP, via Android's broadcast Intent, etc.
- The list of commands or pieces of information that can be sent via this communication interface. You can usually tabulate this.

- For each command/piece of information, what the **structure** of that information is (for example, maybe you use JSON-RPC:), and the **data fields, arguments, etc.** involved. You can define the data structure and fields in a tabular format, for example:

Command: sendMove

Structure: {"jsonrpc": "2.0", "method": "sendMove", "params": {"player": "X", "position": [1,2]}, "id": 1}

Field	Type	Description	Example
"player"	X, O	The player who just made a move.	"X"
"position"	[int, int]	The position at which a move was played, in format [row, col], where [0, 0] is the top-left. Both values must be in the range [0, 2].	[1, 1] (center position)

- Sequences or expected responses. For example, in the documentation of command "A", you can specify that the server will then return an "OK" or "error" response (which is another command documented elsewhere in the section). Another example might be where you have a sequence of commands: for instance, in a challenge-response system, the client might initiate an "auth-request" command, the server sends an "auth-challenge", the client would respond with an "auth-response" containing the correct encrypted response to prove its identity, and the server would send an "auth-failed" or "auth-success". For sequences, it's up to you whether you indirectly define them in the documentation for each command, directly define them in a different sub-section of the "Communication Interfaces" section, define them only in the "Dynamic view" section, or any combination thereof.

3.6 Use Cases and Sequence Diagrams

You should only define use cases and algorithms that are central to how your application works: for example, if your app's entire reason for existence is to choose and connect to the best Wi-Fi network in the area, and periodically re-evaluate this decision, then the Wi-Fi selection algorithm(s) are very important to the app. Do **not** include every little algorithm in your program: those are implementation details you'll work out when you write the code itself, not when you design.

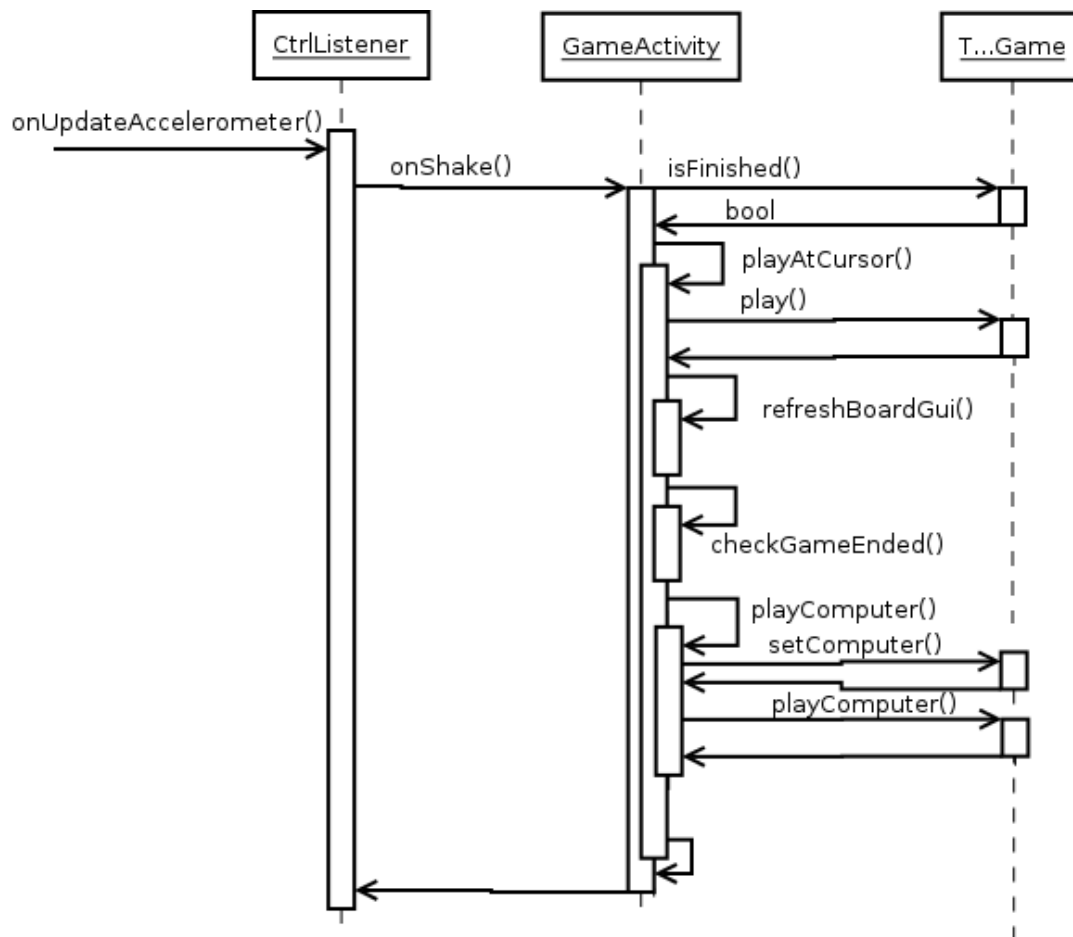


Figure 4: Sequence diagram: player shakes SensorTag and successfully plays a piece

An example of a sequence diagram for SensorTicTagToe is shown in Figure 3 (some of the fine details, like all the getter calls, are omitted as implementation details: it's your call on how much detail to show). Note that sequence diagrams are defined for each different situation where a sequence of method calls happen; examples include user actions, simple use cases, and background actions (e.g. receiving a packet/command from a remote server could use a sequence diagram for how to process that packet/command). This means your sequence

diagrams could become extremely numerous, so you should **pick out just enough scenarios to get the idea of how methods are called and how all the classes are used**, without spending months working on sequence diagrams instead of coding!

Note that a sequence diagram shows **one specific scenario**, so one sequence diagram should not branch out to show different possibilities.

4 Testing

In here you would give a brief explanation on your general goal of testing, for example for this application we need to test the functionality of the TicTacToe game with various wrong input cases. You need to explain how your testing reflects the requirements specified in the PB.

4.1 Test Plan 1: Start a new game

4.1.1 Summary

Requirement ID: GEN-1

The user should be able to start a 2 player game or 1 player game against the AI. The following are intended to be an example and not the full list of test plans and test cases for this application.

4.1.2 Test Cases

Test Case: 1.1		
Pre-Condition: <i>Application launched.</i>		
Steps:	Expected Results	Actual Results
1. click on the 2 players left button	2 players button gets highlighted and AI button becomes grey.	As expected
2. click on Start Game button	The game page should open with two players mode.	
Result: Pass		

Test Case: 1.2		
Pre-Condition: <i>Application lunched.</i>		
Steps:	Expected Results	Actual Results

1. click on the AI right button	2 players button becomes grey and AI button gets highlighted.	As expected
2. click on Start Game button	The game page should open with AI mode.	
Result: Pass		

5 Sprint 2 Backlog

Sprint 2 Goal(s):

- Develop interface to select opponents
- Connect sensor to application

Story ID	Task ID	Task Title	Task Description	Ideal Hours	Status	Comments
HW-1	HW-1.1	Open connection sensor activity.	When the connect button is clicked, go to the sensortag connect activity to choose the sensor.	2 hrs	Completed	
	HW-1.2	Connect to the sensor.	Connect to the sensor after clicking on the available one.	5 hrs	Completed	
GEN-1	GEN-1.1	Game type selection on launch	<i>On first launch, present user with screen to select game type – 1 player or 2 player.</i>	4 hrs	Completed	
	GEN-1.2	Save game type	The selected game type should be saved in user preferences.	3 hrs	In progress	
	GEN-1.3	Future game launch	On future game launches, the saved game type should be used.	2 hrs	Planned	
	GEN-1.4	Game type setting	The user should be able to select game type from settings menu.	2 hrs	Planned	See wireframes for user interface

6 References

- [1] Marc-Alexandre Chan, “SensorTagLib” (software documentation).

Appendix A:

In the appendix, you can include information that is not important for everyone to read (otherwise it should be mentioned in the main document), but someone would be interested in reading it.

For example, screenshots of the mobile app would go into the appendix.